

2교시: Ghost Pending Order - Client 실패와 DB 상태 불일치

Week 3 Day 2 Lesson 2 | frontend와 api 연결 운영 관점
 운영 관점에서의 정상 경로(상대 경로)와 비정상 경로(CORS 오류) 비교

정상 경로: 상대 경로 사용

1 요청 (GET /api/status) (상대 경로)
 2 프록시 전달 /api/status
 3 응답 200 OK (JSON)
 4 응답 전달 200 OK

브라우저 (사용자)

reverse proxy (Nginx / Reverse Proxy (도메인: myapp.com))

api (API 서버 (내부 서비스))

비정상 경로: 외부 API URL 직접 호출 (CORS 발생)

1 잘못된 요청 https://api.example.com/api/status (외부 도메인 직접 호출)
 CORS 오류 응답 차단 (Browser) Access-Control-Allow-Origin 미설정 또는 불일치

문제점

- ✗ CORS 오류로 브라우저가 응답 차단
- ✗ 쿠키, 인증 정보 전송 제한
- ✗ 운영/보안 복잡도 증가

브라우저 개발자 도구 - Network tab

Name	Status	Type	Initiator	Size	Time	Waterfall
/api/status	200	xhr	frontend.js:45	312 B	120 ms	
https://api.example.com/api/status	(blocked by CORS)	xhr	frontend.js:52	0 B	(failed)	

정상 경로 결과

- ✓ 요청: GET /api/status (상대 경로)
- ✓ 상태 코드: 200
- ✓ 응답 수신: 성공
- ✓ 동일 도메인으로 CORS 문제 없음

비정상 경로 결과 (CORS 오류)

- ✗ 요청: https://api.example.com/api/status
- ✗ 상태 코드: (blocked by CORS)
- ✗ 응답 수신: 차단됨
- ✗ 브라우저가 보안상 응답을 차단

Response (200)

```
{
  "status": "ok",
  "uptime": 12345
}
```

Console / Error

```
Access to fetch at 'https://api.example.com/api/status' from origin 'https://myapp.com' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

수업 목표

- Redis 장애 중 order-api가 어떤 불일치 상태를 만들 수 있는지 확인한다.
- client가 실패를 봤는데 DB에는 주문이 남는 상황을 분석한다.
- DB commit과 queue publish 사이의 transaction boundary 문제를 설명한다.
- outbox pattern과 보상 작업이 왜 필요한지 이해한다.

사고 시나리오

Day 1에서는 정상 주문 생성만 확인했다.

Day 2에서는 다음 사고를 본다.

1. order-api가 DB에 주문을 생성한다.
2. DB commit이 끝난다.
3. Redis queue publish 단계에서 실패한다.
4. client는 실패 응답을 본다.
5. 하지만 DB에는 pending 주문이 남아 있다.
6. queue event가 없으므로 worker는 이 주문을 처리하지 못한다.

이것이 ghost pending order다.

실행

```
cd week3/day2/labs/incident-scenarios
./01_ghost_pending_order.sh
```

스크립트가 하는 일:

단계	동작
1	stack을 실행한다
2	Redis를 중지한다
3	같은 request id로 주문 생성을 시도한다
4	client HTTP response를 출력한다
5	DB orders table을 조회한다
6	Redis 복구 후 queue length를 확인한다
7	audit_logs를 조회한다

보야 할 Evidence

Evidence	질문
HTTP response	client는 성공을 봤는가, 실패를 봤는가
orders row	DB에 주문이 남았는가
orders.status	pending인가
Redis LLEN	처리할 event가 있는가
audit_logs	order_created만 있고 order_processed는 없는가

해석

상태	의미
client 503	사용자는 실패로 인식한다
DB pending row 있음	내부에는 업무 흔적이 남았다
queue length 0	worker가 처리할 event가 없다
order_processed 없음	후속 처리가 완료되지 않았다

이 상태에서 사용자가 다시 주문 버튼을 누르면 중복 주문이 생길 수 있다. 운영자는 "실패했으니 아무 일도 없다"고 단정하면 안 된다.

왜 생기는가

현재 교육용 order-api는 단순한 흐름을 가진다.

```
DB insert/commit
-> Redis LPUSH
-> response
```

DB commit과 Redis publish는 하나의 atomic transaction이 아니다. DB는 성공했지만 Redis는 실패할 수 있다.

현실적인 해결책:

해결책	설명
outbox pattern	DB transaction 안에 발행할 event를 같이 저장하고 별도 publisher가 queue로 발행
compensating job	pending 상태가 오래된 주문을 찾아 재발행/취소 처리
idempotency key	사용자가 재시도해도 중복 주문을 만들지 않도록 제어
reconciliation	DB 상태와 queue/audit 상태를 주기적으로 대조

운영 리포트 문장

request_id=... 주문 요청은 Redis 중지 상태에서 client에 실패 응답을 반환했다.
그러나 orders table에는 동일 request_id의 pending row가 생성되어 있었고,
Redis queue에는 처리 대기 event가 없었다.
따라서 client failure와 DB state가 불일치하며, worker가 자동 처리할 수 없는 ghost pending order 상태다.

핵심 포인트

HTTP 실패는 "아무 일도 일어나지 않았다"는 뜻이 아니다.

```
client failure
!= DB rollback
```

MSA에서는 네트워크와 외부 dependency가 분리되어 있으므로 중간 성공/중간 실패 상태를 반드시 고려해야 한다.

Evidence Note

```
# W3D2S2 Ghost Pending Order
- request_id:
- client status:
- DB order row:
- order status:
- queue length after Redis recovery:
- audit rows:
- operational risk:
- remediation:
```